Group Project Guidelines

Web Engineering 2026-2027

October 20, 2026

Contents

1 Introduction **2**
- 1.1 Overview . . . 2
- 1.2 Organisation & Deadlines . . . 2
- 1.3 Starting Code . . . 2

2 Project Agreement **3**
- 2.1 Agreement Budget . . . 3
- 2.2 Possible Functionalities . . . 4
- 2.3 Feature Card . . . 5
- 2.4 AGREEMENT.md . . . 6
- 2.5 TEAM.md . . . 6
- 2.6 How to Submit (2 October) . . . 6

3 Evaluation **7**
- 3.1 Breakdown of Evaluation . . . 7
- 3.2 Per-Feature Grading (50%) . . . 7
  - 3.2.1 Correctness (0 or 5 points) . . . 8
  - 3.2.2 Scalability (0–3.5 points) . . . 8
  - 3.2.3 Engineering (0, 0.75 or 1.5 points) . . . 8
- 3.3 Architecture (20%) . . . 9
- 3.4 Technologies (10%) . . . 9
- 3.5 AI use (10%) . . . 10
- 3.6 Defence (10%) . . . 11
- 3.7 Teacher Reviews (before and at the defence) . . . 11
- 3.8 Rewarding Individual Contributions . . . 12
  - 3.8.1 Ownership (product evidence) . . . 12
  - 3.8.2 Oral at defence (teacher — every student) . . . 12
  - 3.8.3 Integration owner (exactly one per team, named in `TEAM.md`) . . . 13
  - 3.8.4 Labs . . . 13
  - 3.8.5 Peer evaluation (Moodle) . . . 13
- 3.9 Example of Project Scoring . . . 14

4 Milestones **15**

5 Defence 16

1 Introduction

1.1 Overview

- **Project type**: web application focused on **backend development** and **infrastructure**.
- **Team size**: 4 members.

1.2 Organisation & Deadlines

Date (2026)	Milestone	Notes
Fri **18 September**	Group formation	
Fri **2 October**	Project Agreement & Commitment	Moodle zip
Fri **23 October**	Proof of Concept	Moodle tag or zip
Fri **27 November**	Working Prototype	Moodle tag or zip
Thu **10 December**	Presentation to peers	About **5 minutes** per team, in the 2-hour lecture block
Fri **11 December**	Last lecture	
Thu–Fri **17–18 December**	Defence and final submission	Continuous-assessment exam period

1.3 Starting Code

The project uses **Kotlin 2.4** + **Spring Boot 4.1** on **Java 25 LTS**, structured as a **Spring Modulith** application with a hexagonal architecture inside each module.

Provided modules (the seed — weight 0, not scored):

Module	Endpoints	Events
`links`	`POST /api/link`, `GET /{hash}`	Publishes `ShortUrlCreatedEvent` and `ClickLoggedEvent`
`clicks`	—	Owns the `ClickLoggedEvent` type (`links` publishes it on redirect)
`analytics`	`GET /api/stats/{hash}`	Consumes both seed events

The product is the Boot application **plus** the seed's **Docker Compose** stack: two app replicas behind an **nginx** load balancer, backed by **Postgres**. Running the jar on localhost alone is not the product.

```
docker compose up --build                    # load balancer on http://localhost:8080
docker compose --profile load run --rm k6    # basic load test of the whole product
```

Available on GitHub: https://github.com/UNIZAR-30246-WebEngineering/UrlShortener

> **Mandatory use of the Starting Code**
>
> You **must** use the provided starting code unless the teacher explicitly approves an alternative.

2 Project Agreement

2.1 Agreement Budget

Propose **grown features**: new features your team adds on top of the three seed features. Budget rules (locked):

Rule	Value
Weight per feature	**5, 8, 13 or 21** (Fibonacci; maps to ExpectedLevel 1–4)
Budget	Weights sum to **exactly 84** ($= 4 \times 21$); the feature-score denominator is always 84
Grown features	≥ 4, so that each member owns at least one
Event-coupled features	≥ 2
Seed features	Weight **0**; never counted in any rule above

A **grown** feature is a feature card numbered `0004` or higher. The three seed cards (`0001` create short URL, `0002` redirect and click log, `0003` link stats) never count towards the minimums or the budget.

Event-coupled (Y) means the feature publishes or consumes a domain event that crosses a module boundary. Every feature at weight **13** or **21** is event-coupled: Levels 3 and 4 require a **new** event type owned by your team. Reusing `ShortUrlCreatedEvent` or `ClickLoggedEvent` alone does not reach Level 3 or 4.

Why Fibonacci? The weight encodes the **scalability architecture you commit to** (ExpectedLevel), not raw effort:

Weight	Expected Level	Required approach	Properties
5	1	Synchronous Spring MVC	single-threaded · non-durable
8	2	Async threads / coroutines / `@Async`	parallel · non-durable
13	3	In-process events between modules — Spring Modulith `@ApplicationModuleListener`	parallel · module-isolated · non-durable

Weight	Expected Level	Required approach	Properties
21	4	External broker (`--profile broker`)	parallel · module-isolated · durable

Example budget (5 grown features, $\Sigma = 84$, 4 event-coupled):

Feature	Weight	ExpectedLevel	Event-coupled
Async click ingestion (outbox → broker)	21	4	Y
Redirect cache + cross-replica invalidation	21	4	Y
URL Analytics read model	21	4	Y
URL Accessibility Validation	13	3	Y
QR Code Generation	8	2	N

2.2 Possible Functionalities

See the Feature Catalogue for every feature with its owner module, event coupling, and traps.

Validation & Safety — `links` module:

- URL Accessibility Validation
- Google Safe Browsing
- IP and URL Blacklist Check
- URL Expiration
- URL Management (Deactivation)
- Redirection Limits
- QR Code Generation

Observability & Analytics — `analytics` module:

- URL Analytics
- Custom Metrics Aggregation
- Geolocation Service
- Browser and Platform Detection
- Unique Visitor Deduplication

Reliability & Scale — `links` or new module:

- One-Time-Use Links
- Idempotent Link Creation
- Distributed Rate Limiting
- Redirect Cache with Cross-Replica Invalidation

Async & Integration — new module + ADR:

- Async Click Ingestion (outbox → broker)
- Live Click Stream (SSE / WebSocket)

- Webhooks with Retry and Dead-Letter Queue
- Link Preview (OpenGraph Metadata)

Data Operations — `analytics` or `links`:

- CSV Upload
- Async Bulk Export
- GDPR Click Data Anonymisation
- Weighted A/B Redirect

2.3 Feature Card

Each grown feature is one Markdown file in `docs/features/` of your repo, copied from the seed's `docs/features/0000-TEMPLATE.md`. A card has two parts:

- **`## Decision`** — fill by **2 October**: paste the catalogue entry (or write your own in the same shape) and choose a weight.
- **`## Later`** — leave empty on 2 October, then fill it progressively:

`## Later` section	Fill by
ADR	When a choice locks (module, broker, library); by **23 October** at the latest for the feature in your Proof of Concept
Acceptance criteria	**27 November** (Prototype — CI must pass)
Scale evidence	**17–18 December** (Defence)
Qualities self-assessment + AI disclosure	**17–18 December** (Defence)

Example `## Decision` (URL Accessibility Validation, weight 13):

```
**Owner module**: links
**Event-coupled**: N
(synchronous HTTP check - no cross-module event required)

Validates that a target URL returns an acceptable HTTP status code before storing the hash.

**Traps**: servers that reject HEAD (fall back to GET); treating redirects as success;
a synchronous check adding latency to the creation path.

**Technology hints**: `RestTemplate` or `WebClient`; `@ControllerAdvice` + `ProblemDetail`;
circuit breaker for Level 2+.

---

- **Weight:** 13
- **Event-coupled:** Y
```

The owner and event-coupled lines are pasted from the catalogue. The two bullets are your team's commitment. Choosing weight 13 turns event-coupled to **Y**, because Level 3 needs a new event, such as a validation result published to another module.

No personal names on the card. Ownership is recorded in `TEAM.md`.

2.4 AGREEMENT.md

Create `AGREEMENT.md` at the **root of your repo** from the seed's template. It is the cover sheet for the 2 October submission and contains:

- A table of grown feature cards with their owner module, weight, and event-coupled status.
- The budget check: Σ weights $= 84$, grown features ≥ 4, event-coupled ≥ 2.
- The commitment statement.

No personal names, ownership, or integration owner in this file.

2.5 TEAM.md

`TEAM.md` is the only place for personal data: names, UNIZAR emails, git identity (`user.name` / `user.email`), owned features, and the integration owner. Copy it from the seed's `TEAM.md.example`.

> **!** Never commit `TEAM.md`, always zip it
>
> `TEAM.md` is git-ignored. Do **not** commit it, but **do** include it in the zips you upload to Moodle (agreement and final submission).

2.6 How to Submit (2 October)

1. **Feature cards**: one copy of `0000-TEMPLATE.md` per grown feature, with `## Decision` filled and `## Later` empty.
2. **`AGREEMENT.md`**: feature rows (owner module, weight, event-coupled) and budget totals.
3. **`TEAM.md`**: filled from `TEAM.md.example`; kept local.
4. **Upload**: a zip of the working directory, including `TEAM.md`, via Moodle by **2 October**.

> **⚠** Repository Privacy
>
> Keep your GitHub or GitLab repository **private** during development.

3 Evaluation

3.1 Breakdown of Evaluation

The group project is **80%** of continuous evaluation. The **project mark** (0–10) is the weighted sum of five slices, each marked **0–10**:

Slice	Weight	Signal
Features	50%	The product does what the agreement promised
Architecture	20%	Clear modules and how they talk to each other
AI use	10%	GenAI and agents to boost delivery (with disclosure)
Technologies	10%	Correct use of web protocols and standards beyond baseline HTTP
Defence	10%	Presentation to peers, group report, and individual oral checks

Features slice: each grown feature receives a grade from 0 to 10 (see Per-Feature Grading), weighted by its weight. Seed features weigh 0.

$$S = \sum_i (\text{weight}_i \times \text{grade}_i/10) \quad \text{over all grown features}$$

$$\text{features_slice} = 10 \times S/84 \qquad (0\text{–}10)$$

3.2 Per-Feature Grading (50%)

Each grown feature is graded **0–10**:

```
grade = Correctness + Scalability + Engineering
      =  (0 or 5)   +   (0-3.5)   +   (0-1.5)
```

Correctness	Scalability	Engineering	Grade
Pass (5)	Full (3.5)	Full (1.5)	10
Pass (5)	Full (3.5)	None (0)	8.5
Pass (5)	None (0)	Full (1.5)	6.5
Pass (5)	None (0)	None (0)	5
Fail (0)	assessed	assessed	≤ 4 (cap)
Not implemented	—	—	0

3.2.1 Correctness (0 or 5 points)

Pass (5): the feature's acceptance criteria are covered by your team's own automated tests, and CI is green.

Fail (0): otherwise.

The seed's tests (`POST /api/link` → 201, `GET /{hash}` → 307) cover only the seed features; each grown feature needs its own tests. The teacher checks test quality, not just a green CI: weak or tautological tests that would pass without the feature do not count.

3.2.2 Scalability (0–3.5 points)

Scalability measures whether you delivered the architecture level that your weight committed to. Let **ExpectedLevel** be the level mapped from the weight (1–4) and **ImplementedLevel** the level you actually deliver and evidence on the card.

- Weight **5** (ExpectedLevel 1): a correct synchronous implementation scores **3.5**.
- ExpectedLevel $>$ **1**:

$$\text{ScalabilityScore} = \min\left(3.5,\ \frac{\text{ImplementedLevel} - 1}{\text{ExpectedLevel} - 1} \times 3.5\right)$$

For example, a weight-13 feature delivered at Level 2 scores 1.75. Delivering above the ExpectedLevel never scores more than 3.5.

Product-wide load test: every project is run with `docker compose --profile load run --rm k6`. It sends concurrent `POST /api/link` and `GET /{hash}` requests through the load balancer and prints throughput, latency, and failed checks. The test **passes** when the run completes with **zero failed checks** (`create 201`, `redirect 307`). Failing it may lower the Scalability score of every grown feature.

3.2.3 Engineering (0, 0.75 or 1.5 points)

Engineering is not discretionary: two hard gates, then two architecture ticks.

Step 1 — Hard gates (missing either → Engineering = **0**):

1. **AI disclosure** on the feature card: the tools used, or an explicit "No AI assistance".
2. **QA gate**: `./gradlew check` is green (ktlint + detekt + tests + JaCoCo $\geq$ 60% line coverage) and CI is not red.

Step 2 — Architecture ticks (only if both gates pass): an **ADR filed** for the feature, and **`ModularityTests` green**.

Ticks	Engineering
Both	**1.5**
Exactly one	**0.75**

Ticks	Engineering
Neither	**0**

Engineering is verified through automation (`./gradlew check`, CI, `ModularityTests`) and the ADR log, not by reading code manually. Keep CI green throughout.

3.3 Architecture (20%)

The Architecture slice (0–10) rewards engineering rigour in the **module structure**.

What is assessed:

- **Closed modules**: `links`, `clicks`, `analytics`, and any new modules are closed — no module imports another module's `domain`, `application`, or `adapters` packages.
- **Hexagonal enforcement**: each module uses jMolecules stereotypes; `ModularityTests.ensureHexagonal(S` must be green.
- `ApplicationModules.verify()`: the Spring Modulith integrity check must pass in CI.
- **ADRs**: decisions that lock module boundaries, events, Level 4, or specialised technology are recorded in `docs/adr/`, each with the command or test that proves it.
- **Events**: grown features that couple modules do so through events in the module's base package, never by injecting beans from another module's internals.

3.4 Technologies (10%)

The Technologies slice (0–10) rewards **correct use of web protocols and standards** beyond the baseline HTTP that every feature must satisfy.

Baseline (expected of every project; earns **no** Technologies points):

1. **HTTP REST semantics**: correct status codes (`201`, `307`, `400`, `404`, `409`, `410`, `429`), verbs, and a `Location` header on creation.
2. **RFC 9457 Problem Details**: every error response includes `type`, `title`, `status`, `detail`, and `instance`.
3. **Persistence**: Postgres via Spring Data JPA (provided in the Compose stack).
4. **Compose ops**: `docker compose up --build` serves both replicas behind the load balancer, with Actuator readiness checks.

Beyond the baseline, the slice counts the **communication styles** your grown features use. A style counts only when the teacher verifies it is used correctly, not merely present. The rows orient the mark; a project may fall between two rows.

Styles used correctly	Slice mark (0–10)
All three (Call-Return + Event-Based + Data Flow), each in a different grown feature	**10**
Two styles	**6**
One style only (even with several protocols)	**3**

Styles used correctly	Slice mark (0–10)
Baseline HTTP only	**0**

Choose from the catalogue below, or propose another protocol in the same style.

Call-Return (synchronous request/response extensions beyond basic REST):

- **Content negotiation** (`Accept` / `Content-Type`): serve multiple representations from one endpoint — e.g. `image/png` for QR, `text/csv` for bulk export, `application/problem+json` for errors.
- **HTTP Caching** (`Cache-Control`, `ETag`, conditional `GET`): cache redirect targets or analytics payloads at intermediaries; avoid redundant DB reads.
- **Idempotency-Key header**: safe client retry on link creation without duplicate side-effects.
- `multipart/form-data`: file upload for CSV batch processing.
- **OAuth2 / OIDC**: delegated authorisation for management or admin endpoints.
- **gRPC** (HTTP/2 framing): internal RPC when REST overhead is unsuitable; requires a `.proto` schema and bidirectional streaming.
- **GraphQL** (queries + mutations): alternative API style over a single HTTP endpoint; counts here when queries and mutations replace or extend REST endpoints. GraphQL subscriptions count under **Data Flow** instead.
- **HTTP/2 or HTTP/3**: explicitly negotiated upgrade (ALPN) with a measurable benefit for multiplexed or latency-sensitive endpoints.

Event-Based (the server initiates; the client listens or registers a callback):

- **SSE** (`text/event-stream`): server-to-client live push — click stream, analytics feed, or job status.
- **Webhooks**: HTTP callbacks to subscriber-registered URLs, with retry using exponential back-off and `X-Signature-256` verification.

Data Flow (continuous, bidirectional):

- **WebSocket** (`Upgrade: websocket`): persistent bidirectional channel — live analytics dashboard or collaborative features.
- **GraphQL subscriptions** (over WebSocket): real-time event delivery via the GraphQL subscription spec; counts only when the WebSocket transport is correctly implemented (not polling under the hood).

Note: Modulith events, Redis pub/sub, and message brokers count towards **Architecture (20%)**, not here.

3.5 AI use (10%)

GenAI and agents are **allowed** on the project (see the course AI policy). This slice (0–10) rewards an accurate account of how you used them, or an explicit "No AI assistance" when you did not. The rows orient the mark; a project may fall between two rows.

Disclosure quality	Slice mark (0–10)
No AI used — every card states "No AI assistance" explicitly	**10**
Specific AI use — tool, task, and how the output was validated — across multiple features	**10**
AI used; tool named but outcome or validation vague; most cards covered	**6**
Minimal or boilerplate-only disclosure	**3**
Missing disclosure, or disclosure inconsistent with the code	**0**

3.6 Defence (10%)

Assessed from the **peer presentation** (10 December) and the **defence packet** reviewed by the teacher (17–18 December).

What is assessed:

- **Live demo**: `docker compose up` → create, redirect, and stats via **HTTPie**; the scale-evidence commands named on your cards run on the spot, without preparation.
- **Architecture narrative**: module diagram and event flows explained clearly; decisions traceable to ADRs.
- **Group report** (feature cards + ADR log): every card's `## Later` filled (ADR link, acceptance criteria, scale-evidence IDs from the Scalability Guidance, qualities self-assessment, AI disclosure); ADR log coherent.
- **Peer presentation**: clarity, coverage of all grown features, and an honest review against the agreement.

The rows orient the mark; a project may fall between two rows.

Quality	Slice mark (0–10)
Demo runs cleanly, narrative coherent, all cards complete, strong peer presentation	**10**
Minor demo hiccup or one incomplete card; presentation covers the main points	**7**
Demo partially fails, or several cards missing `## Later` sections	**4**
Cannot run the demo, or cards are empty	**0**

3.7 Teacher Reviews (before and at the defence)

- **Review A (automated)**: CI and automated checks — offline, before defence day.
- **Review B (checklist)**: architecture, AI disclosure, and product health — offline, before defence day.
- **Review C (oral)**: **every student** is questioned on 17–18 December, in a ~15-minute slot per team (~2–3 minutes per student plus 5 minutes changeover). The outcome sets the **oral** term of **k** (see below).

3.8 Rewarding Individual Contributions

Baseline (expected of every member; earns no bonus):

- Own **at least one** grown feature (recorded in `TEAM.md`) and make it **Pass**.
- Be ready to explain **any** team feature at the oral.
- Disclose AI use (tools, including GenAI and agents) on the cards you own; help keep the shared product healthy.
- Take part in the defence.

Your individual mark is the project mark multiplied by a factor **k**, capped at **10**:

$$k = 1.0 + \text{ownership} + \text{oral} + \text{integration} + \text{peer} + \text{labs} \quad \text{(clamped to 0.0–1.5)}$$
$$\text{individual mark} = \min(10,\ \text{project mark} \times k)$$

Typical engaged students land at about **k = 1.0–1.25**.

k will be set **0.0** for severe academic-integrity violations.

3.8.1 Ownership (product evidence)

Case	Ownership term
You are named in `TEAM.md` and your owned feature **Passes**	**+0.05**
A second owned feature also **Passes**	**+0.05** more (max **+0.10**)
You are not named, or your owned feature does **not Pass**	**−0.15**

Ownership and oral are separate terms. A feature that does not Pass and an Oral Fail each apply once; neither is counted twice.

3.8.2 Oral at defence (teacher — every student)

Outcome	Meaning	Oral term
Fail	Cannot explain owned or shared/project-level work	**−0.30**
Pass	Coherent at course level (expected)	**0**
Strong	Clear on owned work **and** on one non-owned or project-level question	**+0.10**

3.8.3 Integration owner (exactly one per team, named in `TEAM.md`)

- **Held (+0.10)**: all three hold at the defence: (1) CI is green on the submitted release branch, (2) `docker compose up` works following the README, (3) the required cross-module event path works (demo or test).
- **Dropped (−0.10)**: any one of the three fails.
- Other members: **0**.

3.8.4 Labs

- **−0.05** per missed or late lab (cap **−0.10**).
- **+0.05** only when the lab return explicitly marks **Notable contribution** (peer help captured, extension beyond the brief, or a teaching demo used in class); cap **+0.10** (at most two).
- On time and adequate → **0**. There is no informal "notable".

3.8.5 Peer evaluation (Moodle)

Two waves, same form:

1. **Formative** — after the Working Prototype (**27 November**): detects problems early so the team can get help; it does not affect marks.
2. **Summative** — at the defence (**17–18 December**): feeds **k**.

Form: rate yourself and each teammate on four criteria (contribution, interaction, quality bar, product health) on a Likert scale **1–5**, plus one optional private comment. The mean of your teammates' ratings of you maps linearly to the peer term: **1 → −0.20**, **3 → 0**, **5 → +0.20**. The gap between your self-rating and your peers' ratings is **feedback only** (no automatic penalty).

What counts as free-riding in this project (use these when rating peers — look for a **pattern**, not one bad week):

Signal	Free-rider looks like…	Not free-riding…
Contribution	No grown feature in their name, or a named feature that never **Passes**; teammates repeatedly finish "their" work	Owns a smaller feature that **Passes** and can explain it
Interaction	Ghosts planning, reviews, and integration; decisions happen without them	Quiet in chat but shows up for commits, reviews, and defence prep
Quality bar	Ships noise others must rewrite; skips tests/CI; empty or fake AI disclosure	Needs help, asks early, incorporates feedback
Product health	Breaks shared pipelines and leaves them broken; no care for Compose/CI/docs beyond "their" files	Integration-heavy role with a clear owned feature that still **Passes**

Signal	Free-rider looks like…	Not free-riding…
Oral / cards	Cannot explain own or teammates' features; feature cards show no real trail of their work	Nervous at the oral but coherent on the owned feature and team work

A single missed deadline or a weak first draft is **not** free-riding. A free-rider **systematically** takes the team mark while avoiding ownership, defence, and shared product care.

Corroboration: a **positive** peer term applies from the summative wave alone. A **negative** peer term below **−0.05** applies only when corroborated by at least one of: the 27 November formative wave, an Oral Fail at the defence, or feature-card evidence. Without corroboration, the negative peer term stops at **−0.05**.

Teacher override: the teacher may adjust or discard peer results when ratings look collusive (near-identical means with empty comments), when a free-rider flag lacks corroboration, or when other exceptional conditions appear. Peer inputs stay confidential; each student receives the breakdown of their own **k**.

3.9 Example of Project Scoring

Group ZZZZZ — 4 members, 5 grown features (Σ weights = 84):

Feature	Weight	Correctness	Scalability	Engineering	Grade
Async click ingestion (Level 4 of 4)	21	5	3.50	1.50	**10.00**
URL Analytics read model (Level 4 of 4)	21	5	3.50	1.50	**10.00**
Redirect cache (Level 4 of 4)	21	5	3.50	0.75	**9.25**
URL Accessibility Validation (Level 2 of 3)	13	5	1.75	0.75	**7.50**
QR Code Generation (not implemented)	8	0	0	0	**0.00**

$$S = (21 \times 10.00 + 21 \times 10.00 + 21 \times 9.25 + 13 \times 7.50 + 8 \times 0.00)/10 \approx 71.2$$

$$\text{features_slice} = 10 \times 71.2/84 \approx 8.47$$

Slice marks (0–10):

Slice	Mark	Weight	Contribution
Features	8.47	50%	4.24
Architecture	8.0	20%	1.60
AI use	7.5	10%	0.75

Slice	Mark	Weight	Contribution
Technologies	6.5	10%	0.65
Defence	7.0	10%	0.70
Project mark			**7.94**

Individual marks (factor k):

Member	Owns	Ownership	Oral	Integration	Peer	Labs	k	Individual mark
Alice	Async click ingestion	+0.05	+0.10	+0.10	+0.10	0	**1.35**	**10.00**
Bob	URL Analytics, Redirect cache	+0.10	0	0	0	0	**1.10**	**8.73**
Carol	URL Accessibility Validation	+0.05	0	0	−0.05	−0.05	**0.95**	**7.54**
Dan	QR Code Generation	−0.15	−0.30	0	−0.10	−0.05	**0.40**	**3.18**

Alice is the integration owner. $7.94 \times 1.35 = 10.72$, so her mark stops at the cap of 10. Bob owns two features that Pass, so he earns the maximum ownership term. Dan's feature did not Pass and he could not explain it at the oral: the ownership Fail and the Oral Fail each apply once. His −0.10 peer term goes below −0.05 because the Oral Fail corroborates it.

> ⚠ Correction (20 October 2026)
>
> The 8 October brief weighted this example at Features 25% and Architecture 10%, with a project mark of **5.02**. The weights above match the breakdown: Features **50%**, Architecture **20%**. The project mark is **7.94**. Individual marks are Alice **10.00** (capped), Bob **8.73**, Carol **7.54**, and Dan **3.18**.

4 Milestones

- **Project Agreement & Commitment — 2 October**
  - Feature cards (`## Decision` only), `AGREEMENT.md`, and a local `TEAM.md` — see How to Submit.
  - Upload a zip of the working directory (including `TEAM.md`) via Moodle.
- **Proof of Concept — 23 October**
  - Modules wired; at least **one grown feature with an event** published by one module and consumed by another, covered by a test.

  - ADR filed for the feature that event belongs to.
  - `docker compose up` starts without errors.
  - Submit a repo tag or zip via Moodle.

- **Working Prototype — 27 November**
  - At least **one grown feature delivered at Level 3 or 4** (weight 13 or 21) and passing CI.
  - Acceptance criteria filled on every card.
  - The k6 load test passes (zero failed checks).
  - Submit a repo tag or zip via Moodle.

- **Defence and Final Project Submission — 17–18 December**
  - All grown features complete; every card's `## Later` filled.

5 Defence

Deliverables:

- **Code** (zip via Moodle): application, tests, and `TEAM.md` (git-ignored — include it in the zip; do not commit it).
- **Defence packet** — bring it to your oral slot:
  - Feature cards (`docs/features/`) filled from `0000-TEMPLATE.md`: `## Decision` (catalogue entry, weight, event-coupled) and `## Later` (ADR link, acceptance criteria, scale-evidence IDs, qualities self-assessment, AI disclosure).
  - ADR log (`docs/adr/`) filled from `docs/adr/0000-TEMPLATE.md`. Each decision's **Evidence (defence)** checklist names the code link, the test or command that proves it (`./gradlew test`, k6, `docker compose …`), and a diagram or Modulith doc snippet when the structure changed.
  - Live demo on the Compose load balancer (`http://localhost:8080`):

```
docker compose up --build
http POST localhost:8080/api/link url=https://example.com
http GET  localhost:8080/<hash> --follow
http GET  localhost:8080/api/stats/<hash>
```

- **Peer presentation** (**10 December**, about **5 minutes** per team in the 2-hour lecture block):
  - High-level architecture: module diagram and event flows.
  - Work distribution and feature ownership.
  - Integration and testing approach (`ModularityTests`, acceptance tests).
  - Objective review against the agreement cards.